

AI Engineer Take-Home Assignment

GitHub repo link, with a README

Overview

Build a single **configurable agent** that can act as one of three financial personas, pull sector-specific context from a database you build, and answer questions both from a human (via a Streamlit UI or equivalent) and via an API call — using MCP (Model Context Protocol) as the integration layer.

What you'll build

1. Persona-configurable agent

The agent must support switching between three personas via a config or parameter, each with a distinct voice, priorities, and analytical lens:

Persona	Lens
Mutual Fund Analyst	Long-only, benchmark-relative, focuses on sustainable growth, valuation vs. index, portfolio fit
Equity Analyst	Fundamentals-driven — earnings, margins, competitive positioning, price targets
PE Analyst	Deal/ops lens — cash flow, leverage capacity, operational levers, exit potential

Same underlying data, three different framings of the answer. We want to see the persona meaningfully change *how* the agent reasons and communicates, not just a cosmetic tone change.

2. Sector context, sourced by you

Pick and scrape/compile data for **3 sectors** (e.g. Tech, Retail, Manufacturing, Logistics — pick any 3) from public sources (company filings, press releases, public financial data sites, news, etc.). Load this into a real database (SQLite is fine) with a schema you design.

- The agent must query this DB live to answer questions — no hardcoding facts into prompts.

- Sector must also be switchable via config/params, independent of persona (any persona × any sector = 9 valid combos).
- Document your schema and sourcing in the README, including any known data-quality caveats.

3. MCP-based tool exposure

Expose the DB query capability (and any other tools the agent uses) as **MCP endpoints/tools**, and have the agent consume them via MCP rather than direct function calls. We want to see you actually use MCP as the protocol boundary, not just import a DB client inline.

4. Dual interface: human + API

The same agent logic must be reachable two ways:

- **Human-facing:** a simple chat/Q&A interface in **Streamlit / equivalent**, with persona + sector selectable in the UI.
- **API:** a REST (or equivalent) endpoint that accepts a query + persona + sector and returns a structured JSON response — usable by another system, not just a person.

Both paths should hit the same underlying agent — don't build two separate implementations.

Example usage

Persona: **PE Analyst**, Sector: **Logistics** Query: *"Which companies in this sector look like attractive buyout targets based on the data you have?"*

The agent should ground its answer in the specific companies/data in your DB for that sector, reasoned through a PE lens (leverage, ops improvement potential, exit multiples) — and give a different answer/framing than the same question asked as an MF Analyst.

More sample queries

Use these to sanity-check your own agent before submitting — and expect us to run similar ones during review. Good answers should differ meaningfully by persona even when the sector and underlying data are identical.

Cross-persona, same question (try all 3 personas on this one):

Sector: **Tech** Query: *"Is this sector a good place to be putting money to work right now?"*

- MF Analyst should frame this relative to benchmark/index exposure and growth durability.

- Equity Analysts should ground the answer in earnings trends, margins, and valuation multiples.
- PE Analysts should talk about deployable capital, entry multiples, and exit timelines.

Persona: MF Analyst, Sector: Retail

"Which of these companies would fit a long-term core holding versus a name I should avoid?"

Persona: Equity Analyst, Sector: Manufacturing

"Walk me through the margin profile of the companies in your data — who's improving and who's under pressure?"

Persona: PE Analyst, Sector: Tech

"If I had to pick one company here to take private, which would it be and what's the operational thesis?"

Data-grounding stress test (any persona/sector):

"What's the most recent headcount or hiring signal you have for [a specific company in your DB]?" This should force a real DB lookup — a good way for you (and us) to catch hallucination vs. genuine retrieval.

Out-of-scope test (any persona/sector):

"What do you think about [a company not in your dataset]?" The agent should clearly indicate it has no data on this company rather than fabricating an answer — we're checking for honest scope-awareness, not fluent bluffing.

API-specific test:

POST a query with `persona=equity_analyst`, `sector=logistics`, and a question — confirm the JSON response includes the answer plus enough structure (e.g. sources/companies referenced, confidence, or similar) to be consumed programmatically, not just a raw text blob.

Submission checklist

- GitHub repo (public or shared with us) with clear README and setup instructions
- `.env.example` for any required keys (no real keys committed)
- Sample DB or a script to (re)build it from source data

- Short write-up (in README, ~1 page) on: schema decisions, MCP design, and one thing you'd improve with more time
- Loom/video walkthrough (optional but appreciated) — 3–5 min demo of both interfaces

Notes

- We care about how you think and structure the system more than sector-data completeness — a few dozen well-sourced records per sector is plenty.
- Use whatever LLM provider you're comfortable with; note your choice and any API keys we'd need to run it.
- If you run out of time, prioritize a working MCP + dual-interface skeleton over exhaustive persona polish — tell us in the README what you'd have done next.